PIAZZA DEI MESTIERI – TORINO

IeFP – Operatore Informatico | Prima Annualità (Classe B22-19-20270)

PROGRAMMAZIONE 1

Fondamenti di Logica, Algoritmi e Sviluppo Software

Manuale Didattico Ufficiale del Corso di Laboratorio (80 ore)

Docente: Daniele Cerrina
Email: daniele.cerrina@immaginazioneelavoro.it
Anno Formativo: 2026 / 2027

Programmazione 1 – Fondamenti e Laboratorio
Percorso triennale IeFP per la Qualifica di Operatore Informatico.
© 2026–2027 Piazza dei Mestieri / Immaginazione e Lavoro.
Docente: Daniele Cerrina.

Documento didattico in continuo aggiornamento durante l'anno formativo.
Compilato con LaTeX e pdflatex.

Presentazione del Libro

Questo volume rappresenta il punto di riferimento e il libro di testo per il corso di **Programmazione 1** (80 ore di laboratorio) presso la Piazza dei Mestieri di Torino.

L'approccio adottato è fortemente pratico e incrementale: ogni lezione svolta in aula e in laboratorio viene consolidata e integrata all'interno di questo testo, fornendo agli allievi:

- Le spiegazioni teoriche chiare dei concetti chiave.
- Gli esempi concreti e analogie vicine all'esperienza quotidiana e videoludica.
- Gli schemi logici ed esercizi svolti e da svolgere.
- Il percorso di sviluppo del **gioco guida annuale** votato dalla classe.

Come consultare questo documento: Il documento è provvisto di un **indice ipertestuale e cliccabile**. Cliccando su qualsiasi capitolo o sezione nell'Indice Generale o nei segnalibri del lettore PDF, verrai indirizzato direttamente alla pagina corrispondente.

Indice Generale

Presentazione del Libro ii

I Pensiero Computazionale e Algoritmi 1

1 Introduzione agli Algoritmi e Pensiero Computazionale 2
1.1 Che cos'è un Algoritmo? . . . 2
1.1.1 Esempi dal Quotidiano e dal Mondo Digitale . . . 3
1.2 Le Quattro Proprietà Fondamentali . . . 3
1.3 Esercizio Guidato: Attraversamento Pedonale . . . 5

2 La Struttura Sequenziale 6
2.1 Ripartiamo dalla Definizione . . . 6
2.2 Il Modello Fondamentale I-P-O . . . 6
2.3 La Sequenza Pura . . . 7
2.4 Esempi Svolti . . . 7
2.5 Esercizi di Laboratorio su Carta . . . 9
2.6 I 4 Errori Tipici della Sequenza . . . 10

3 Le Condizioni e i Bivi nel City Builder Marziano 11
3.1 Perché un Gestionale ha Bisogno di Bivi? . . . 11
3.2 Cos'è una Condizione? La Logica Booleana . . . 11
3.3 Gli Operatori Relazionali . . . 12
3.4 La Struttura SE ... ALLORA ... ALTRIMENTI . . . 12
3.4.1 Il Pattern del City Builder: Verifica e Spesa . . . 13
3.5 Le Forme del Flow Chart . . . 13
3.6 Esempio Svolto: la Cupola Idroponica . . . 14
3.6.1 Passo 1: Lo Schema I-P-O . . . 14
3.6.2 Passo 2: Il Flow Chart . . . 14
3.6.3 Passo 3: Lo Pseudocodice . . . 14
3.7 Esercizi di Laboratorio: Disegna il Flow Chart . . . 16
3.8 Soluzioni Commentate . . . 17
3.9 Checklist del Flow Chart . . . 19

4 Capire il Problema: Leggere da Programmatore 20
4.1 Perché Leggere Prima di Risolvere? . . . 20
4.2 Lo Schema I-P-O come Filtro di Lettura . . . 20
4.3 Il Metodo del Programmatore in 5 Passi . . . 21
4.4 Il Kit dell'Evidenziatore: Colori e Simboli . . . 21
4.5 Il Dizionario: Parole che Diventano Confronti . . . 22
4.6 Le 4 Trappole del Testo . . . 22

4.7 Esempio Svolto: Il Turno dell'Ossigeno 23
4.8 Le 5 Sfide del Programmatore 24
4.9 Soluzioni Commentate 26
4.10 Errori Tipici e Checklist del Programmatore 28

Guida Rapida alla Sintassi **30**

Part I

Pensiero Computazionale e Algoritmi

1

Introduzione agli Algoritmi e Pensiero Computazionale

"Un algoritmo deve essere visto per essere creduto, ma soprattutto per essere compreso: è il ponte invisibile che trasforma un problema disordinato in una soluzione esatta."

Riferimento Lezione: 21 Settembre 2026 – Prima Lezione di Laboratorio
Competenze chiave: Comprensione del modello Input/Output, identificazione delle proprietà di un algoritmo, formulazione di istruzioni non ambigue.

1.1 Che cos'è un Algoritmo?

Nel linguaggio comune siamo abituati a risolvere problemi ogni giorno senza pensare alle singole azioni che compiamo. Quando però vogliamo affidare la risoluzione di un problema a un computer (o spiegare a un'altra persona come agire senza margini d'errore), abbiamo bisogno di un metodo rigoroso.

Definizione di Algoritmo

Un **algoritmo** è una sequenza finita e ordinata di istruzioni elementari, chiare e non ambigue, la cui esecuzione porta alla risoluzione di un determinato problema in un tempo finito (altrimenti parleremo di loop, argomento di cui ci occuperemo più avanti).

In informatica ogni problema viene formalizzato attraverso il modello fondamentale di elaborazione:

- **Input (Dati di ingresso):** Rappresenta l'informazione o la condizione iniziale fornita all'algoritmo. È ciò che "entra" nel processo.
- **Elaborazione:** L'insieme ordinato di istruzioni elementari applicate ai dati di ingresso.
- **Output (Risultato):** Il dato finale o l'azione risultante al termine dell'elaborazione. È ciò che "esce" dal processo.

INPUT (Dati / Problema) $\longrightarrow$ [**ALGORITMO**]
$\longrightarrow$ **OUTPUT** (Risultato / Soluzione)

1.1.1 Esempi dal Quotidiano e dal Mondo Digitale

Gli algoritmi non appartengono solo al codice sorgente: li sperimentiamo quotidianamente in molteplici contesti:

Algoritmi nella vita reale

1. **Tutorial per costruire una casa in Minecraft:** Un elenco ordinato di azioni (1. Raccogli 64 blocchi di legno, 2. Costruisci una base 6×6, 3. Innalza i muri per 4 blocchi, 4. Posiziona la porta e le torce). Se inverti i passaggi (ad esempio posizionare il tetto prima dei muri), l'obiettivo fallisce.
2. **Ricetta di cucina (es. Torta al cioccolato):** L'elenco degli ingredienti rappresenta l'**Input**; la sequenza dei passaggi (pesare, mescolare, infornare a 180 gradi per 35 minuti) costituisce l'**Algoritmo**; il dolce pronto è l'**Output**.
3. **Istruzioni di montaggio (LEGO o mobili IKEA):** Diagrammi e numeri sequenziali guidano la costruzione partendo dai singoli pezzi fino all'oggetto completo.

1.2 Le Quattro Proprietà Fondamentali

Non ogni elenco di frasi o istruzioni può essere definito un algoritmo valido. Per essere considerato formalmente **corretto**, un algoritmo deve soddisfare quattro caratteristiche essenziali:

1. **Finitezza:** L'algoritmo deve necessariamente terminare dopo aver eseguito un numero finito di istruzioni. Un procedimento che gira all'infinito senza mai giungere a una conclusione non è un algoritmo.
2. **Definitezza (Non ambiguità):** Ogni istruzione deve essere univoca e comprensibile senza alcuna interpretazione soggettiva. Frasi come "mescola un po' " o "aspetta qualche secondo" non sono adatte a un algoritmo informatico: bisogna specificare "mescola per 3 minuti a velocità 2" o "attendi 5 secondi".
3. **Efficienza:** L'algoritmo deve risolvere il compito impiegando una quantità *ragionevole* di tempo e di memoria.

   Nota

   Nel corso di quest'anno la priorità assoluta sarà comprendere la **logica** e raggiungere il **corretto funzionamento**; la misurazione scientifica dell'efficienza (complessità computazionale) sarà affrontata negli anni successivi.

4. **Generalità:** L'algoritmo non deve risolvere un singolo caso specifico e isolato, ma deve essere in grado di risolvere **qualsiasi problema appartenente a quella specifica classe**.

Il Principio della Generalità

Il nostro obiettivo annuale è imparare a risolvere qualsiasi problema della categoria che stiamo affrontando.

Esempio pratico: Se progettiamo un algoritmo per calcolare la media aritmetica di 7 numeri, l'algoritmo deve funzionare correttamente sia con la sequenza $[1, 2, 3, 4, 5, 6, 7]$, sia con $[9, 8, 7, 6, 5, 4, 3]$, sia con qualsiasi altra combinazione di 7 valori numerici, inclusi numeri decimali o negativi.

1.3 Esercizio Guidato: Attraversamento Pedonale

Proviamo a scrivere un algoritmo esatto per un problema che affrontiamo ogni giorno.

Attraversare una strada con un semaforo

Scrivere l'algoritmo passo-passo, il più rigoroso e aderente possibile alle quattro proprietà esaminate, per risolvere il seguente problema:
Problema: Mi trovo sul marciapiede davanti alle strisce pedonali e devo attraversare la strada in presenza di un semaforo pedonale. Cosa faccio?

Algoritmo con Scelta Condizionale e Ripetizione

Ecco la formalizzazione dell'algoritmo divisa in istruzioni elementari:

1. **Osservazione:** Guardo il semaforo pedonale.
2. **Controllo condizionale:**
   - **SE** il semaforo è **VERDE**:
     (a) Inizio ad attraversare a passo costante sulle strisce pedonali.
     (b) Faccio un passo dopo l'altro **FINCHÉ** non ho raggiunto il marciapiede opposto.
     (c) Fine dell'algoritmo (Obiettivo raggiunto con successo!).
   - **ALTRIMENTI** (il semaforo è rosso o giallo):
     (a) Resto fermo sul marciapiede di partenza in sicurezza.
     (b) Aspetto che il semaforo cambi stato.
     (c) **Torno al punto 1** (ripeto l'osservazione).

Attenzione all'ambiguità e ai cicli infiniti!

- Se dimentichiamo la condizione "**FINCHÉ** non arrivo dall'altra parte", una persona programmata come un computer continuerebbe a camminare dritto all'infinito anche dopo aver superato la strada!
- Se omettiamo di dire cosa fare nel caso il semaforo sia rosso, il calcolatore si bloccherebbe per mancanza di istruzioni.
- Notiamo come questo algoritmo funzioni con qualsiasi strada che abbia un semaforo in cui c'è almeno il verde.

2

La Struttura Sequenziale

> *"Prima i muri, poi il tetto: in un algoritmo l'ordine dei passi non è un dettaglio, è la soluzione stessa."*

Riferimento Lezione: 28 Settembre 2026 – Seconda Lezione di Laboratorio
Competenze chiave: Scomposizione di un problema nel modello Input–Elaborazione–Output, scrittura di sequenze lineari di istruzioni elementari, riconoscimento degli errori tipici di ordine e di ambiguità.

2.1 Ripartiamo dalla Definizione

Nel capitolo precedente abbiamo definito l'algoritmo come una sequenza **finita** e **ordinata** di istruzioni elementari, chiare e **non ambigue**. Rileggiamo la definizione mettendo l'accento su tre parole:

- **Sequenza ordinata:** l'ordine cronologico dei passi è vincolante. Scambiare due istruzioni altera il risultato o blocca l'esecuzione.
- **Istruzioni elementari:** passi atomici che l'esecutore (umano o macchina) sa compiere senza dubbi né interpretazioni soggettive.
- **Problema e soluzione:** l'algoritmo parte da una condizione iniziale (**Input**) e produce un risultato concreto (**Output**).

2.2 Il Modello Fondamentale I-P-O

Ogni elaborazione informatica si fonda su tre momenti, che d'ora in poi chiameremo **schema I-P-O** (*Input – Process – Output*):

INPUT ⟶ **ELABORAZIONE** ⟶ **OUTPUT**

1. **Input (dati in ingresso):** le informazioni fornite al sistema, come valori digitati dall'utente, misure o prezzi.

2. **Elaborazione (Process):** la serie ordinata di calcoli e trasformazioni eseguite dal calcolatore passo dopo passo.
3. **Output (dati in uscita):** il risultato restituito, come un testo a video, un totale stampato o un disegno completato.

Le Quattro Proprietà, in Breve

Un algoritmo corretto è **finito** (termina sempre), **definito** ("aggiungi 5 grammi di zucchero", non "un po' di zucchero"), **generale** (risolve ogni problema della sua classe, non un solo caso con dati fissi) ed **efficiente**. In prima superiore la priorità è al 100% sulla correttezza logica.

2.3 La Sequenza Pura

Nelle prime settimane di laboratorio lavoriamo esclusivamente sulla **sequenza lineare**, la più semplice delle strutture di controllo.

La Struttura Sequenziale

In una **sequenza** le istruzioni vengono eseguite rigidamente **dall'alto verso il basso**, una dietro l'altra:

- non ci sono bivi né ripetizioni: ogni riga viene eseguita **esattamente una volta**;
- **l'ordine temporale è tassativo**: non si può usare o elaborare un dato prima di averlo letto o calcolato.

2.4 Esempi Svolti

Costruire un Rifugio di Terra 3x3 in Minecraft

Input (inventario): 33 blocchi di terra, 1 porta di legno, 1 torcia, 1 letto, 1 fornace, 1 banco da lavoro.
Output: rifugio 3×3 chiuso, illuminato e arredato per superare la notte.

1. Posiziona il perimetro della base con 8 blocchi di terra, lasciando un vuoto per la porta.
2. Innalza il secondo livello dei muri (8 blocchi).
3. Innalza il terzo livello dei muri (8 blocchi).
4. Copri il soffitto con 9 blocchi di terra: il tetto 3×3 è chiuso.
5. Posiziona la porta di legno per sigillare l'ingresso.
6. Attacca la torcia su una parete interna (la luce impedisce la comparsa dei mostri).
7. Posiziona il letto in un angolo per impostare il punto di rinascita.
8. Posiziona la fornace.
9. Posiziona il banco da lavoro.

10. Chiudi la porta e dormi: notte superata!

Perché l'ordine è legge? Se provi a piazzare la torcia o il tetto prima di aver innalzato i muri, non hai un supporto solido su cui posizionarli. Se metti il letto prima di chiudere i muri, un mostro ti elimina prima dell'alba.

Preparare un Toast Farcito alla Piastra

Input: 2 fette di pane da toast, 2 fette di formaggio, 1 fetta di prosciutto cotto, piastra elettrica, 1 piatto.
Output: toast caldo e dorato con formaggio fuso, pronto sul piatto.

1. Appoggia le due fette di pane sul tagliere pulito.
2. Adagia la prima fetta di formaggio sulla fetta di pane inferiore.
3. Aggiungi la fetta di prosciutto cotto sopra il formaggio.
4. Adagia la seconda fetta di formaggio sopra il prosciutto.
5. Chiudi il toast sovrapponendo la seconda fetta di pane.
6. Accendi la piastra elettrica e attendi che sia calda.
7. Posiziona il toast sulla piastra e chiudi il coperchio.
8. Attendi 3 minuti per la tostatura.
9. Apri la piastra e preleva il toast con una paletta.
10. Adagia il toast sul piatto e servilo caldo.

Perché l'ordine è fondamentale? Se metti il formaggio sulla piastra prima del pane, il formaggio brucia. Se provi a tostare il pane prima di accendere la piastra, resta crudo.

2.5 Esercizi di Laboratorio su Carta

Esercizio 1: La Media di 3 Voti

Uno studente ha ottenuto 3 voti in informatica: **7**, **8** e **6**.
Scrivi sul quaderno, in italiano chiaro, la sequenza lineare che calcola e restituisce la media esatta. Indica prima l'**Input**, poi i passi dell'elaborazione e infine l'**Output**.

Esercizio 1: La Media di 3 Voti

Input: i tre voti (7, 8 e 6).

1. Leggi il primo voto, il secondo voto e il terzo voto.
2. Calcola la somma dei tre voti: $7 + 8 + 6 = 21$.
3. Dividi la somma per il numero dei voti: $21 \div 3 = 7$.
4. Mostra la media ottenuta.

Output: la media, cioè 7.
Per rispettare la **generalità** la sequenza deve funzionare con qualsiasi terna di voti: si scrive "somma dei tre voti diviso 3", non "21 diviso 3".

Esercizio 2: Il Robot Disegnatore

Un robot su una lavagna digitale comprende solo due istruzioni:

- **"Avanza di 100 passi tracciando una linea dritta"**;
- **"Gira a destra di 90 gradi"**.

Scrivi la sequenza esatta di 8 istruzioni che fa disegnare al robot un quadrato chiuso e lo fa tornare orientato come in partenza.

Esercizio 2: Il Robot Disegnatore

1. Avanza di 100 passi tracciando una linea dritta.
2. Gira a destra di 90 gradi.
3. Avanza di 100 passi tracciando una linea dritta.
4. Gira a destra di 90 gradi.
5. Avanza di 100 passi tracciando una linea dritta.
6. Gira a destra di 90 gradi.
7. Avanza di 100 passi tracciando una linea dritta.
8. Gira a destra di 90 gradi.

Le istruzioni 1–4 tracciano due lati; le 5–8 chiudono il quadrato. L'ultima rotazione (passo 8) non disegna nulla, ma riporta il robot nella direzione di partenza: senza di essa il quadrato sarebbe completo, ma la consegna no.

2.6 I 4 Errori Tipici della Sequenza

Gli Errori da Evitare

1. **Dare per scontato l'ovvio:** saltare un passaggio intermedio che per noi è banale ma per la macchina è indispensabile.
2. **Passi invertiti:** calcolare il totale prima di aver letto i dati in ingresso.
3. **Istruzioni ambigue:** usare termini vaghi come "un po"' invece di numeri e misure esatte.
4. **Soluzioni cablate (*hardcoded*):** scrivere il risultato già calcolato a mente anziché il procedimento generale.

3

Le Condizioni e i Bivi nel City Builder Marziano

"Un gestionale diventa intelligente nel momento in cui impara a guardare il magazzino prima di spendere."

Riferimento Lezione: 5 Ottobre 2026 – Terza Lezione di Laboratorio
Progetto Guida Annuale: Mars City Builder (Lite)
Competenze chiave: Formulazione di condizioni booleane, uso degli operatori relazionali, struttura di selezione `SE ... ALLORA ... ALTRIMENTI`, disegno di Flow Chart con le forme standard.

3.1 Perché un Gestionale ha Bisogno di Bivi?

Con la **struttura sequenziale** (Capitolo 2) le istruzioni vengono eseguite una dietro l'altra, senza mai deviare. Immaginiamo però di programmare il nostro **city builder su Marte** usando solo la sequenza.

Il giocatore clicca sul pulsante *"Costruisci Cupola Idroponica"*. Se il programma non controlla prima il magazzino, piazza l'edificio sulla mappa anche se il giocatore ha 0 unità di metallo: il bilancio scende a -50 e la partita si rompe.

La Selezione

Un programma diventa capace di reagire alla situazione quando sa **osservare lo stato delle risorse prima di agire**. Questa struttura logica si chiama **selezione** (o bivio condizionale): in base a una condizione, il programma sceglie quale strada percorrere.

3.2 Cos'è una Condizione? La Logica Booleana

Una **condizione** è un'espressione di confronto che ammette solo **due** risultati possibili: **VERO** oppure **FALSO**. Non esistono vie di mezzo: o hai abbastanza metallo o non ce l'hai. Questa logica a due valori si chiama **logica booleana**, dal matematico George Boole.

Condizioni nel City Builder

- `metallo >= 50`: con metallo = 70 è **VERO**; con metallo = 30 è **FALSO**.
- `energia_prodotta >= energia_richiesta`: con $100 \geq 80$ è **VERO**; con $60 \geq 80$ è **FALSO**.
- `coloni == 0`: se i coloni sono 0 è **VERO** (colonia deserta); se sono 12 è **FALSO**.

Frasi che NON Sono Condizioni

- *"La colonia è bella?"*: è soggettiva, il computer non ha gusti.
- *"Costruisci qualche casa"*: è vaga, quante case esattamente?

3.3 Gli Operatori Relazionali

Per confrontare risorse e parametri della colonia usiamo sei operatori relazionali. Nella tabella supponiamo che il giocatore abbia `crediti = 50`.

Simbolo	Matematica	Significato	Esempio	Risultato
`>`	$>$	strettamente maggiore	`crediti > 20`	**VERO**
`<`	$<$	strettamente minore	`crediti < 10`	**FALSO**
`>=`	$\geq$	maggiore o uguale	`crediti >= 50`	**VERO**
`<=`	$\leq$	minore o uguale	`crediti <= 40`	**FALSO**
`==`	$=$	uguale a	`crediti == 50`	**VERO**
`!=`	$\neq$	diverso da	`crediti != 50`	**FALSO**

La Trappola Numero 1: = contro ==

- `=` (**assegnazione**) è un ordine: prende un valore e lo memorizza nella variabile. `crediti = 100` deposita 100 crediti nel conto.
- `==` (**confronto**) è una domanda: chiede se i due valori sono identici. `crediti == 100` restituisce VERO o FALSO senza toccare la memoria.

3.4 La Struttura SE ... ALLORA ... ALTRIMENTI

La selezione divide il cammino del programma in due rami **mutuamente esclusivi**: si esegue il primo oppure il secondo, mai entrambi nello stesso turno. Alla fine i due rami si ricongiungono e il programma prosegue.

```
SE <condizione> ALLORA
    // ramo VERO
    istruzione_A1
    istruzione_A2
ALTRIMENTI
    // ramo FALSO
    istruzione_B1
FINE SE
```

```
// il programma si ricongiunge qui
```

La Selezione a Una Via

Se nel caso FALSO non serve fare niente di speciale, il ramo `ALTRIMENTI` si può omettere. Esempio: un bonus di fine turno facoltativo.

```
SE colonia_felice == VERO ALLORA
    crediti = crediti + 20
FINE SE
salva_partita()
```

3.4.1 Il Pattern del City Builder: Verifica e Spesa

Ogni volta che il giocatore ordina una costruzione, l'algoritmo segue sempre la stessa sequenza logica:

1. **Verifica risorse:** le scorte coprono il costo? (`SE risorsa >= costo`)
2. **Aggiornamento magazzino (se VERO):** piazza l'edificio e scala il costo (`risorsa = risorsa - costo`).
3. **Feedback a schermo:** mostra un messaggio chiaro, di conferma o di risorse insufficienti.

3.5 Le Forme del Flow Chart

Il **Flow Chart** (diagramma a blocchi) disegna la logica di un algoritmo. Ogni forma geometrica ha un significato standard:

Forma	Funzione	Esempio
	**Inizio e fine** del flusso	`INIZIO / FINE`
	**Input e output**: lettura dei dati e stampa a video	`LEGGI metallo`
	**Operazioni e calcoli**: assegnazioni e modifiche della memoria	`metallo = metallo - 50`
	**Condizione e bivio**: due uscite obbligatorie, SÌ e NO	`metallo >= 50 ?`

Le 3 Regole d'Oro del Flow Chart

1. **Frecce direzionali:** collegano i blocchi dall'alto verso il basso e non restano mai aperte.
2. **Etichette sul rombo:** ogni rombo ha due frecce uscenti, etichettate **SÌ** (VERO) e **NO** (FALSO).
3. **Punto di ricongiunzione:** i due rami si riuniscono prima dell'output finale e di

```
    FINE.
```

3.6 Esempio Svolto: la Cupola Idroponica

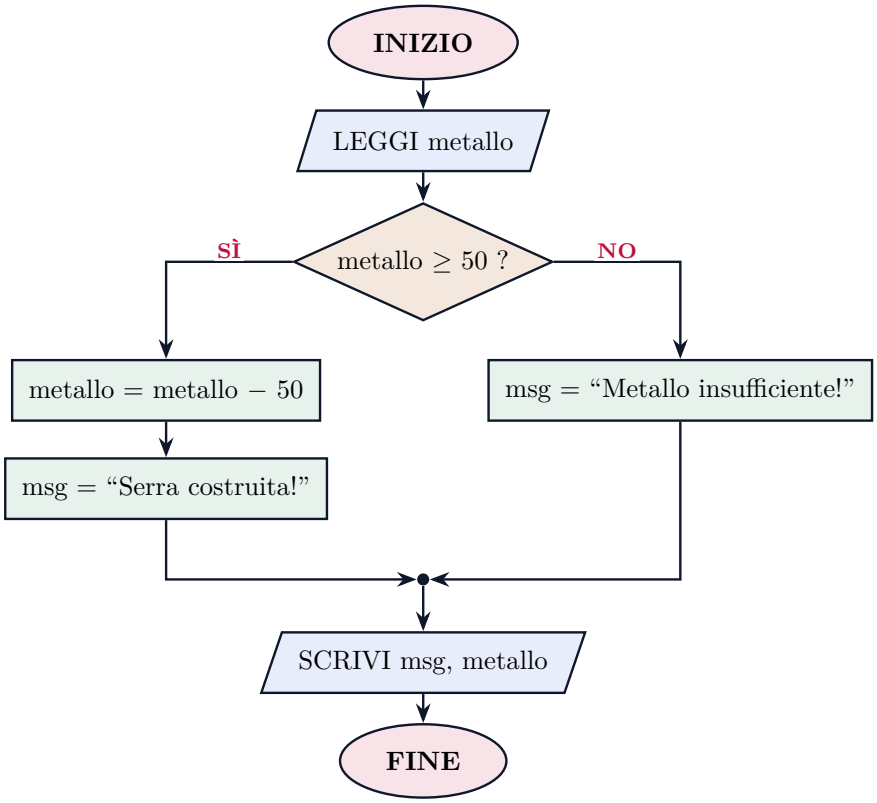
Il Testo del Problema

Ogni Cupola Idroponica costa **50 unità di metallo**. L'algoritmo legge il `metallo_disponibile` nel magazzino centrale. Se il metallo è **pari o superiore a 50**, scala 50 unità dal magazzino e imposta il messaggio *"Serra costruita con successo!"*; altrimenti non costruisce nulla e imposta il messaggio *"Metallo insufficiente per la Serra!"*. Alla fine mostra il messaggio e il metallo rimasto.

3.6.1 Passo 1: Lo Schema I-P-O

- **Input:** `metallo_disponibile` (numero intero ≥ 0).
- **Elaborazione:** se `metallo_disponibile >= 50` scala il costo e prepara il messaggio di successo; altrimenti prepara il messaggio di errore.
- **Output:** il messaggio e il saldo aggiornato del metallo.

3.6.2 Passo 2: Il Flow Chart



3.6.3 Passo 3: Lo Pseudocodice

```
// 1. Acquisizione del dato dal magazzino
LEGGI metallo_disponibile

// 2. Controllo di fattibilita' e spesa
SE metallo_disponibile >= 50 ALLORA
    metallo_disponibile = metallo_disponibile - 50
    messaggio = "Serra costruita con successo!"
ALTRIMENTI
    messaggio = "Metallo insufficiente per la Serra!"
FINE SE

// 3. Notifica al giocatore
SCRIVI messaggio
SCRIVI "Metallo rimasto in magazzino: ", metallo_disponibile
```

3.7 Esercizi di Laboratorio: Disegna il Flow Chart

Per ciascun esercizio disegna sul quaderno, con penna e righello, il **Flow Chart completo** rispettando le forme standard.

Esercizio 1: Il Bilancio Energetico della Colonia

Il sole rosso di Marte sta calando dietro i crateri di Tharsis. I coloni rientrano nelle cupole, accendono i riscaldatori e i purificatori d'aria passano alla modalità notturna. Come responsabile dell'energia, devi verificare se la carica dei pannelli solari basterà per tutta la notte.

- Se l'energia prodotta copre o supera il fabbisogno della base, la rete è stabile e la colonia passa la notte in sicurezza.
- Se la produzione è insufficiente, scatta l'allarme blackout: bisogna staccare macchinari e fabbriche.
- Il programma mostra a schermo lo stato della rete.

Esercizio 2: Nuovi Coloni e Razioni di Cibo

I retrorazzi della navetta "Ares Shuttle" sollevano una nube di polvere rossa: sono sbarcati nuovi coloni. Prima di aprire i portelloni, l'amministratore consulta i silos, sapendo che ogni colono consuma esattamente una cassa di cibo a turno.

- Se le casse bastano (o avanzano) per tutti i coloni, il cibo viene distribuito e le scorte diminuiscono della quantità consumata.
- Se le scorte non bastano per tutti, la distribuzione non avviene e scatta l'allarme carestia.
- Il programma mostra l'esito e il saldo finale delle scorte.

Esercizio 3: L'Indice di Felicità della Colonia

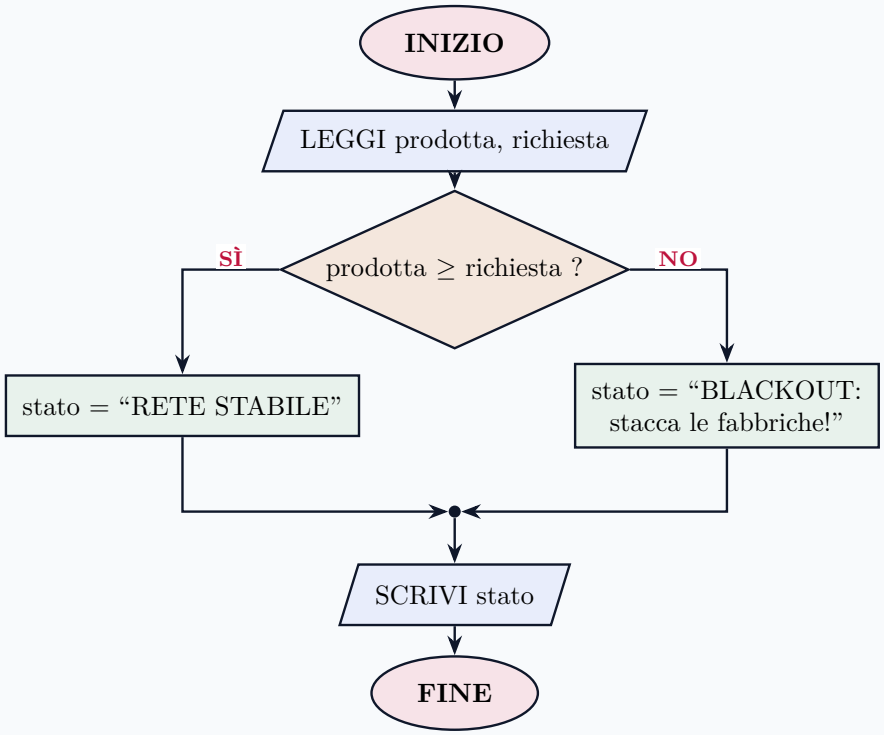
È venerdì sera nella cupola centrale e si tiene l'assemblea cittadina. I sensori sociali registrano l'indice di gradimento della popolazione, da 0 a 100.

1. **Colonia fiorente:** se il gradimento raggiunge o supera 75, la colonia incassa un bonus di 50 crediti.
2. **Rivolta:** se il gradimento scende sotto 40, scatta lo sciopero e la produzione si blocca.
3. **Colonia stabile:** in tutti i casi intermedi (da 40 a 74) la vita prosegue regolarmente.

Il programma comunica lo stato finale. **Suggerimento:** usa due rombi a cascata, il secondo sul ramo NO del primo.

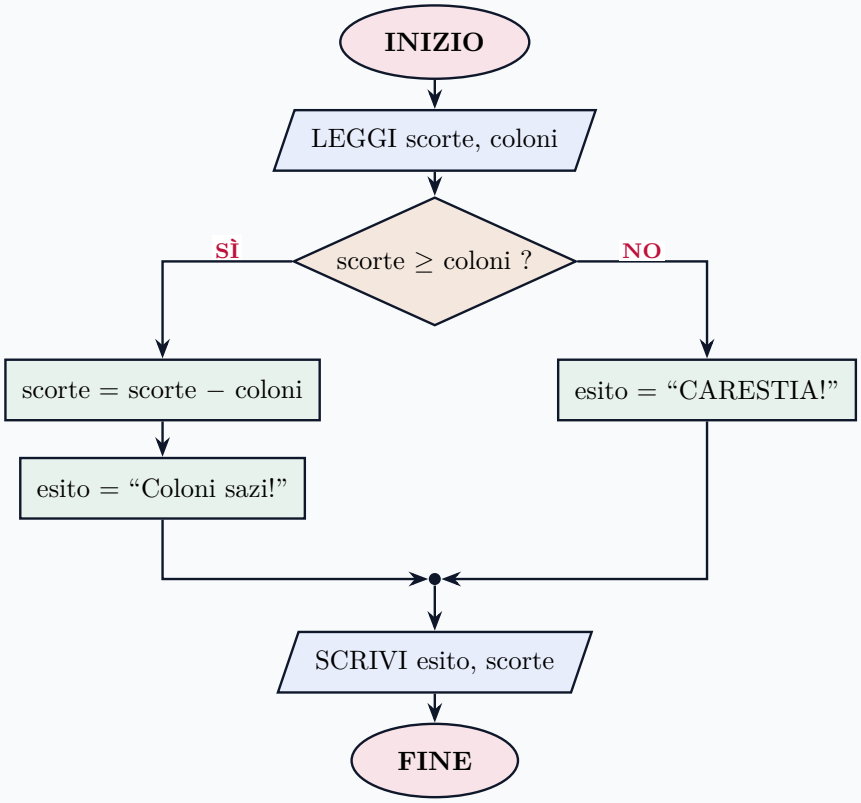
3.8 Soluzioni Commentate

Esercizio 1: Il Bilancio Energetico



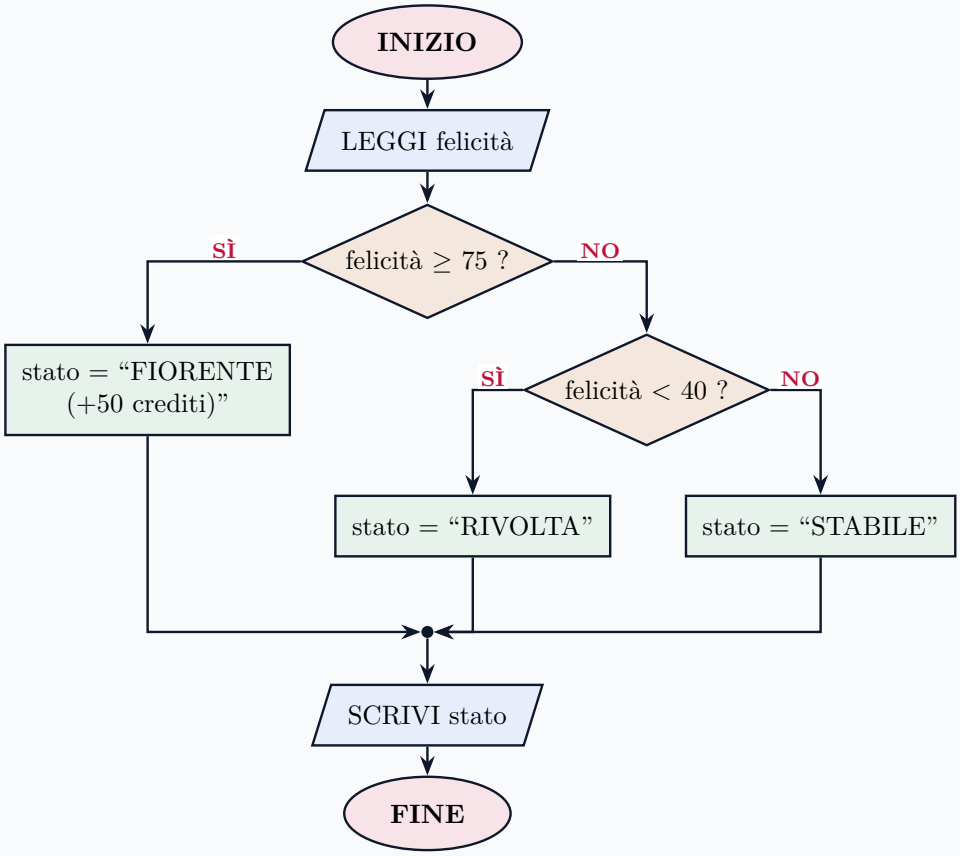
“Copre o supera” diventa $\geq$: con produzione esattamente uguale al fabbisogno la rete è stabile.

Esercizio 2: Nuovi Coloni e Razioni



La sottrazione delle scorte sta **solo** nel ramo SÌ: se il cibo non basta, il silos resta com'era.

Esercizio 3: L'Indice di Felicità



Il secondo rombo viene raggiunto solo quando la felicità è sotto 75: per questo il ramo NO finale copre esattamente l'intervallo da 40 a 74, senza bisogno di un terzo confronto.

3.9 Checklist del Flow Chart

Prima di Consegnare il Disegno

- ☐ Ho usato i **parallelogrammi** per ogni `LEGGI` e `SCRIVI`?
- ☐ Ho usato i **rettangoli** per calcoli e assegnazioni?
- ☐ Ogni **rombo** ha due frecce uscenti con le etichette **SÌ** e **NO**?
- ☐ I rami di ogni bivio si **ricongiungono** prima dell'output finale?
- ☐ Il diagramma ha un solo **INIZIO** e una sola **FINE**?

4

Capire il Problema: Leggere da Programmatore

> *"Il computer fa esattamente quello che gli diciamo. Se leggiamo male il testo, il bug nasce prima ancora di disegnare il primo blocco."*

Riferimento Lezione: 12 Ottobre 2026 – Quarta Lezione di Laboratorio
Progetto Guida Annuale: Mars City Builder (Lite)
Competenze chiave: Lettura attiva del testo di un problema, separazione tra dati, richiesta, regole e rumore, traduzione delle parole in confronti, compilazione della scheda I-P-O e verifica del caso limite.

In questo capitolo non scriviamo né codice né pseudocodice: impariamo a **capire** il problema, che è il passo che viene prima di tutti gli altri.

4.1 Perché Leggere Prima di Risolvere?

Il Bug Nato da Due Parole

Il testo dice: *"L'allarme deve suonare quando restano **al massimo 10** unità di ossigeno."*
Il programmatore frettoloso capisce: *"L'allarme suona quando restano **meno di 10** unità."*

Con esattamente 10 unità il testo vuole l'allarme ($10 \leq 10$ è VERO), il programma invece tace ($10 < 10$ è FALSO). Nessun errore di calcolo: solo una lettura sbagliata.

4.2 Lo Schema I-P-O come Filtro di Lettura

Lo schema I-P-O (Capitolo 2) non serve solo a progettare: è anche un **filtro** per leggere. A ogni frase del testo chiediamo in quale casella va.

	Domanda da farsi	Cosa cercare nel testo
INPUT	Cosa mi **dai**?	I dati noti: numeri, quantità, misure.
ELABORAZIONE	Quali **regole**?	Calcoli e condizioni che collegano dati e risultato.
OUTPUT	Cosa mi **chiedi**?	Il risultato da trovare o il messaggio da mostrare.

Il Rumore

Tutto quello che non entra in nessuna casella è **rumore**: nomi, orari, colori, emozioni. Rende bella la storia, ma il computer non lo usa. A volte il rumore è perfino un **numero**.

4.3 Il Metodo del Programmatore in 5 Passi

1. **Leggi due volte.** La prima volta per capire la storia, la seconda per cercare i dati.
2. **Evidenzia** con i colori o con i simboli del kit (Sezione 4.4).
3. **Riformula** con una sola frase: *"Devo capire se..."*.
4. **Compila la scheda I-P-O** sul quaderno: riformulazione, input, elaborazione, output.
5. **Prova a mano** con i numeri del testo e poi con il **caso limite**.

Regola d'Oro

Nessuna soluzione finché non hai completato i 5 passi.

4.4 Il Kit dell'Evidenziatore: Colori e Simboli

Chi ha gli evidenziatori usa **colore e simbolo insieme**. Chi non li ha usa **solo il simbolo**, con una penna qualsiasi: ogni simbolo si disegna a mano in un secondo.

Cosa cerco	Colore	Come appare	Simbolo a mano
Dato (Input)	blu	40 unità	**cerchio**: cattura il dato
Richiesta (Output)	rosso	serra riparata?	**doppia sottolineatura**: il risultato
Regola	giallo	se basta	**onda**: il bivio, la strada cambia
Rumore	—	~~il tecnico Rossi~~	**barrato**: lo butto via
Dato nascosto	blu	★ 7 giorni	★ **a margine** + il dato
Dato mancante	—	? metallo	**?** **a margine** + la domanda

Quando una regola contiene un confronto, lo scriviamo accanto: se il cibo basta ≥.

4.5 Il Dizionario: Parole che Diventano Confronti

Parola nel testo	Confronto	Esempio	La soglia è...
almeno	$\geq$	"almeno 50 di metallo"	compresa
al massimo	$\leq$	"al massimo 6 coloni"	compresa
più di / supera	$>$	"supera 120 km/h"	esclusa
meno di / sotto	$<$	"meno di 40 punti"	esclusa
fino a	$\leq$	"resiste fino a 120 km/h"	compresa
basta / è sufficiente	$\geq$	"il cibo basta"	compresa
esattamente / pari a	$=$	"esattamente 10 coloni"	solo quella
diverso da / tranne	$\neq$	"tutti tranne il rover 2"	esclusa

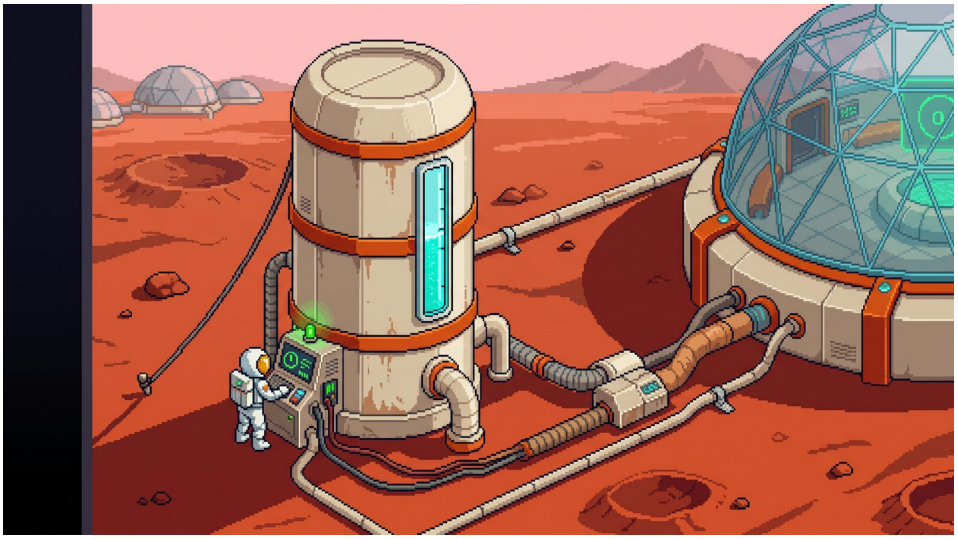
Il Caso Limite

Prova sempre con il numero **esatto** della soglia (50, 120, 10...). È lì che "almeno" e "più di" danno risultati opposti, ed è lì che si nasconde il bug.

4.6 Le 4 Trappole del Testo

1. **Il rumore:** nomi, orari e perfino numeri che non servono (*"la serra ha perso il 40% delle piante"*).
2. **Il dato nascosto:** il testo non lo scrive, ma serve (*"per una settimana"* $\rightarrow$ ★ 7 giorni).
3. **Le parole-trappola:** "almeno" e "più di" sembrano uguali, ma sul caso limite danno risultati opposti.
4. **Il dato mancante:** un'informazione indispensabile non c'è. Non si inventa: si scrive **?** e la domanda da fare.

4.7 Esempio Svolto: Il Turno dell'Ossigeno



Il Testo

Sono le 6:40 del mattino nella Colonia Alpha-1 e la sirena del cambio turno ha appena suonato. Il tecnico Rossi, arrivato su Marte da appena tre settimane, controlla il pannello del generatore. Ogni colono consuma 3 unità di ossigeno al giorno. Il serbatoio contiene 40 unità e nella colonia vivono 12 coloni. Se l'ossigeno del serbatoio basta per tutta la giornata, il pannello deve mostrare "Respirate tranquilli"; altrimenti deve far scattare l'allarme rosso. Cosa deve mostrare il pannello oggi?

Il Testo Annotato e la Scheda I-P-O

~~Sono le 6:40 del mattino nella Colonia Alpha-1 e la sirena del cambio turno ha appena suonato. Il tecnico Rossi, arrivato su Marte da appena tre settimane,~~ controlla il pannello del generatore. Ogni colono consuma 3 unità di ossigeno al giorno. Il serbatoio contiene 40 unità e nella colonia vivono 12 coloni. Se l'ossigeno del serbatoio basta $\geq$ per tutta la giornata, il pannello deve mostrare "Respirate tranquilli"; altrimenti deve far scattare l'allarme rosso. Cosa deve mostrare il pannello oggi?

Riformulo	*"Devo capire se l'ossigeno basta per tutti i coloni di oggi."*
Input	ossigeno nel serbatoio = **40** • coloni = **12** • consumo a testa = **3**
Elaborazione	fabbisogno = coloni $\times$ 3 basta? $\rightarrow$ serbatoio $\geq$ fabbisogno
Output	"Respirate tranquilli" **oppure** allarme rosso
Prova a mano	$12 \times 3 = 36 \rightarrow 40 \geq 36$ **VERO** $\rightarrow$ "Respirate tranquilli" Caso limite: serbatoio = 36 $\rightarrow$ $36 \geq 36 \rightarrow$ basta ancora!

4.8 Le 5 Sfide del Programmatore

Modalità: **a coppie**. Su ogni testo evidenzia con colori o simboli. Sul quaderno scrivi la frase *"Devo capire se... "*, la scheda I-P-O e la prova a mano con il caso limite.

Sfida 1: La Tempesta di Sabbia

Il radar meteo della base Tharsis segnala una tempesta di sabbia in arrivo da nord-ovest, a 380 chilometri di distanza. Le raffiche sollevano la polvere rossa fino a 2 chilometri di altezza. La comandante Okafor, al suo secondo anno di missione, posa il caffè liofilizzato e ordina di controllare gli scudi. Gli scudi della cupola resistono a venti fino a 120 km/h; il radar stima che la tempesta soffierà a 135 km/h. Se il vento supera la resistenza degli scudi, i coloni devono rifugiarsi nel bunker; altrimenti possono restare nelle cupole. Dove devono andare i coloni?

Sfida 2: Razioni per la Settimana

La navetta cargo Ares-3 è atterrata ieri sera sulla pista n. 2, ma il comandante vuole sapere se il cibo in magazzino basta per una settimana intera, senza aspettare il prossimo rifornimento. Nella colonia vivono 10 coloni e ognuno mangia 2 razioni al giorno. Nel magazzino ci sono 150 razioni. Le razioni preferite dai coloni sono quelle alla soia affumicata. Se il cibo non basta, bisogna ordinare un rifornimento urgente dalla Terra. Serve il rifornimento?

Sfida 3: La Fabbrica di Mattoni

Il consiglio della colonia ha votato all'unanimità: è arrivato il momento di accendere la nuova fabbrica di mattoni. Per partire, la fabbrica ha bisogno di almeno 50 unità di energia libera. La centrale solare produce 200 unità, mentre gli edifici già accesi ne consumano 150. Se l'energia libera è meno di 50, la fabbrica resta spenta e i muratori tornano a giocare a carte nella mensa; altrimenti la fabbrica si accende. La fabbrica si accende?

Sfida 4: Il Nuovo Quartiere

Con l'ultima navetta sono arrivati 9 nuovi coloni che non hanno ancora un letto. Il sindaco Hernández, che ama i nomi poetici, ha già deciso di chiamare il nuovo quartiere "Valle dell'Alba". Ogni modulo abitativo ospita al massimo 6 coloni e costa 80 unità di metallo. Se il metallo in magazzino basta, si costruiscono tutti i moduli necessari; altrimenti i nuovi coloni dormono nella palestra della base. Quanti moduli servono? Si possono costruire?

Sfida 5: Il Meteorite sulla Serra

Durante la lunga notte marziana, mentre la colonia dorme, un piccolo meteorite colpisce la serra idroponica. L'ingegnera capo Lindqvist, svegliata dall'allarme, corre a controllare in tuta pressurizzata: la serra ha perso il 40% delle piante e 4 pannelli di vetro sono in frantumi. Riparare un pannello costa 30 unità di metallo. Se il metallo in magazzino è almeno pari al costo della riparazione, la serra viene riparata subito; se invece non basta, la serra resta chiusa fino al prossimo rifornimento. In magazzino ci sono 100 unità di

metallo. La serra verrà riparata?

La Scheda I-P-O da Copiare sul Quaderno

Riformulo	Devo capire se .....
Input	(nome = valore)
Elaborazione	(calcoli e confronti)
Output	
Prova a mano	(+ caso limite)

4.9 Soluzioni Commentate

Sfida 1: La Tempesta di Sabbia — trappola: il rumore

~~Il radar meteo della base Tharsis segnala una tempesta di sabbia in arrivo da nord-ovest, a 380 chilometri di distanza. Le raffiche sollevano la polvere rossa fino a 2 chilometri di altezza. La comandante Okafor, al suo secondo anno di missione, posa il caffè liofilizzato~~ e ordina di controllare gli scudi. Gli scudi della cupola resistono a venti fino a $\leq$ 120 km/h; il radar stima che la tempesta soffierà a 135 km/h. Se il vento supera la resistenza degli scudi $>$, i coloni devono rifugiarsi nel bunker; altrimenti possono restare nelle cupole. Dove devono andare i coloni?

Riformulo	*"Devo capire se il vento è più forte di quanto reggono gli scudi."*
Input	resistenza scudi = **120 km/h** • velocità del vento = **135 km/h**
Elaborazione	"supera" → vento > resistenza
Output	"Tutti nel bunker" **oppure** "Restate nelle cupole"
Prova a mano	$135 > 120$ **VERO** → tutti nel bunker! Caso limite: vento = 120 → $120 > 120$ FALSO → gli scudi reggono ("fino a").

Anche i 380 km e i 2 km sono rumore: sono numeri, ma non entrano in nessuna casella.

Sfida 2: Razioni per la Settimana — trappola: il dato nascosto

~~La navetta cargo Ares-3 è atterrata ieri sera sulla pista n. 2,~~ ma il comandante vuole sapere se il cibo in magazzino basta $\geq$ per una settimana intera ★ 7 giorni, senza aspettare il prossimo rifornimento. Nella colonia vivono 10 coloni e ognuno mangia 2 razioni al giorno. Nel magazzino ci sono 150 razioni. ~~Le razioni preferite dai coloni sono quelle alla soia affumicata.~~ Se il cibo non basta $<$, bisogna ordinare un rifornimento urgente dalla Terra. Serve il rifornimento?

Riformulo	*"Devo capire se 150 razioni coprono 7 giorni per tutti i coloni."*
Input	razioni = **150** • coloni = **10** • razioni a testa al giorno = **2** • ★ giorni = **7**
Elaborazione	fabbisogno = coloni $\times$ 2 $\times$ 7 basta? → razioni $\geq$ fabbisogno
Output	"Nessun rifornimento" **oppure** "Rifornimento urgente!"
Prova a mano	$10 \times 2 \times 7 = 140$ → $150 \geq 140$ **VERO** → nessun rifornimento.

Senza il ★ si calcola $10 \times 2 = 20$ e si arriva alla stessa risposta per il motivo sbagliato: con 130 razioni il calcolo corretto fa scattare il rifornimento, quello sbagliato no.

Sfida 3: La Fabbrica di Mattoni — trappola: le parole-trappola

~~Il consiglio della colonia ha votato all'unanimità: è arrivato il momento di~~ accendere la nuova fabbrica di mattoni. Per partire, la fabbrica ha bisogno di almeno $\geq$ 50 unità

di energia libera. La centrale solare produce 200 unità, mentre gli edifici già accesi ne consumano 150 **libera = 200 − 150**. Se l'energia libera è meno di < 50, la fabbrica resta spenta ~~e i muratori tornano a giocare a carte nella mensa~~; altrimenti la fabbrica si accende. La fabbrica si accende?

Riformulo	*"Devo capire se l'energia che avanza arriva ad almeno 50."*
Input	produzione = **200** • consumo = **150** • soglia = **50**
Elaborazione	energia libera = produzione − consumo "almeno" → libera ≥ 50 (il contrario: libera < 50)
Output	"Fabbrica accesa" **oppure** "Fabbrica spenta"
Prova a mano	200 − 150 = 50 → 50 ≥ 50 **VERO** → si accende! Chi ha letto "più di 50": 50 > 50 FALSO → fabbrica spenta. Bug!

Sfida 4: Il Nuovo Quartiere — trappola: il dato mancante

Con l'ultima navetta sono arrivati 9 nuovi coloni che non hanno ancora un letto. ~~Il sindaco Hernández, che ama i nomi poetici, ha già deciso di chiamare il nuovo quartiere "Valle dell'Alba".~~ Ogni modulo abitativo ospita al massimo ≤ 6 coloni e costa 80 unità di metallo. Se il metallo in magazzino basta ≥ **? quanto metallo c'è?**, si costruiscono tutti i moduli necessari; altrimenti i nuovi coloni dormono nella palestra della base. Quanti moduli servono? Si possono costruire?

Riformulo	*"Devo capire quanti moduli servono per 9 coloni e se ho il metallo per costruirli."*
Input	coloni = **9** • posti a modulo = **6** • costo = **80** metallo in magazzino = **? MANCA**
Elaborazione	1 modulo = 6 posti (non basta) → servono **2 moduli** costo = 2 × 80 = 160 → metallo ≥ 160
Output	numero di moduli (2) + "Costruiti" **oppure** "Tutti in palestra"
Domanda da fare	Non posso rispondere! Chiedo al sindaco: **quanto metallo c'è in magazzino?** Se fossero 200 → 200 ≥ 160 → 2 moduli costruiti.

La risposta corretta è: *"Servono 2 moduli, ma non posso dire se si possono costruire."* Inventare il dato mancante è un errore, anche se il risultato sembra plausibile.

Sfida 5: Il Meteorite sulla Serra — tutte le trappole

~~Durante la lunga notte marziana, mentre la colonia dorme,~~ un piccolo meteorite colpisce la serra idroponica. ~~L'ingegnera capo Lindqvist, svegliata dall'allarme, corre a controllare in tuta pressurizzata: la serra ha perso il 40% delle piante~~ e 4 pannelli di vetro sono in frantumi. Riparare un pannello costa 30 unità di metallo **costo = 4 × 30**. Se il metallo in magazzino è almeno pari ≥ al costo della riparazione, la serra viene

riparata subito; se invece non basta $<$, la serra resta chiusa fino al prossimo rifornimento. In magazzino ci sono 100 unità di metallo. La serra verrà riparata?

Riformulo	*"Devo capire se il metallo basta per riparare tutti i pannelli rotti."*
Input	pannelli rotti = **4** • costo a pannello = **30** • metallo in magazzino = **100**
Elaborazione	costo riparazione = pannelli × 30 "almeno pari" → metallo ≥ costo
Output	"Serra riparata" **oppure** "Serra chiusa"
Prova a mano	$4 \times 30 = 120 \rightarrow 100 \geq 120$ **FALSO** → serra chiusa. Caso limite: metallo = 120 → 120 ≥ 120 → riparata!

Il 40% è un **numero esca**: sembra importante, ma non entra in nessun calcolo. Domanda bonus: quanto metallo manca? $120 - 100 = 20$ unità.

4.10 Errori Tipici e Checklist del Programmatore

Gli Errori Più Comuni

- Partire a risolvere dopo una sola lettura.
- Cerchiare **tutti** i numeri, anche il 40% e i 380 km.
- Dimenticare il dato nascosto (i 7 giorni).
- Leggere "almeno" come "più di".
- Inventare un dato mancante invece di chiederlo.

La Checklist da Usare Prima di Ogni Flow Chart

- ☐ Ho letto il testo **due volte**?
- ☐ Ho **barrato** il rumore, anche i numeri che non servono?
- ☐ Ogni dato ha un **nome** e un **valore**?
- ☐ Ho trovato la **richiesta** (doppia sottolineatura)?
- ☐ Ho cercato i **dati nascosti** (★) e quelli **mancanti** (?)?
- ☐ Ho tradotto le regole in **confronti** ($\geq$, $>$, $\leq$, $<$, $=$, $\neq$)?
- ☐ Ho provato con i numeri del testo e con il **caso limite**?

Biglietto d'Uscita: Il Rover Esploratore

Il rover Curiosity-2, verniciato di arancione, consuma 5 unità di energia per ogni uscita e ha la batteria carica a 18 unità. Il comandante vuole sapere se il rover può fare almeno 3 uscite prima di ricaricarsi.

Biglietto d'Uscita: Il Rover Esploratore

Il rover ~~Curiosity-2, verniciato di arancione,~~ consuma 5 unità di energia per ogni uscita e ha la batteria carica a 18 unità. Il comandante vuole sapere se il rover può fare almeno $\geq$ 3 uscite prima di ricaricarsi.

Riformulo	*"Devo capire se 18 unità bastano per 3 uscite."*
Input	batteria = **18** • consumo a uscita = **5** • uscite = **3**
Elaborazione	energia necessaria = uscite $\times$ 5 $\rightarrow$ batteria $\geq$ necessaria
Output	"Sì, può partire" **oppure** "No, prima ricarica"
Prova a mano	$3 \times 5 = 15 \rightarrow 18 \geq 15$ **VERO** $\rightarrow$ sì, può fare 3 uscite.

Guida Rapida alla Sintassi

In questa appendice finale verranno raccolti durante l'anno i promemoria di sintassi, le parole chiave e i comandi più utilizzati in laboratorio.

Operatori di Confronto

Simbolo	Matematica	Significato
`>`	$>$	strettamente maggiore
`<`	$<$	strettamente minore
`>=`	$\geq$	maggiore o uguale
`<=`	$\leq$	minore o uguale
`==`	$=$	uguale a (confronto, non assegnazione!)
`!=`	$\neq$	diverso da

Forme del Flow Chart

- **Ovale:** `INIZIO` e `FINE`.
- **Parallelogramma:** input e output (`LEGGI`, `SCRIVI`).
- **Rettangolo:** calcoli e assegnazioni (`metallo = metallo - 50`).
- **Rombo:** condizione con due uscite, **SÌ** e **NO**.

Il Kit dell'Evidenziatore

Cosa cerco	Come appare	Simbolo a mano
Dato	12 coloni	cerchio
Richiesta	cosa mostra?	doppia sottolineatura
Regola	se basta	onda
Rumore	il tecnico Rossi	barrato
Dato nascosto	★ 7 giorni	★ a margine
Dato mancante	? quanto metallo?	? a margine

Parole che Diventano Confronti

Parola	Confronto	Parola	Confronto
almeno, basta	$\geq$	più di, supera	$>$
al massimo, fino a	$\leq$	meno di, sotto	$<$
esattamente, pari a	$=$	diverso da, tranne	$\neq$